```
#include <linux/fb.h>
#include <linux/init.h>
#include <linux/pci.h>

static int __init ourfb_init(void)
{
     return pci_register_driver(&ourfb_driver);
}
static void __exit ourfb_exit(void)
{
     pci_unregister_driver(&ourfb_driver);
}
module_init(ourfb_init);
module_exit(ourfb_exit);
MODULE_LICENSE("GPL");
```

So now we have a very basic module with the required *init* and *exit* function, but we have not yet defined *'ourfb_driver'* which we are using in the code to register in the PCI subsystem.

```
static struct pci_driver ourfb_driver = {
     .name =   "ourfb",
     .id_table =  ourfb_pci_tbl,
     .probe =  ourfb_pci_init,
     .remove = ourfb_pci_remove,
};
```

In the driver structure, we have defined only the required part. If you want to use power management in your driver code and provide *Suspend* and *Resume* functionality to your hardware, then you can define that in the driver. But here we are leaving them out as they are optional.

In *ourfb_pci_tbl*, we have to mention the vendor ID and the device ID of the hardware for which you are writing the driver.

```
static struct pci_device_id ourfb_pci_tbl[] = {
     { PCI_VENDOR_ID_XXX, PCI_DEVICE_ID_XXX, PCI_ANY_ID, PCI_
ANY_ID },
     { 0 }
};
MODULE_DEVICE_TABLE(pci, ourfb_pci_tbl);
```

We are assuming that *PCI_VENDOR_ID_XXX* and *PCI_DEVICE_ID_XXX* are defined elsewhere with the correct vendor and device ID. This table should always be null terminated. To explain *MODULE_DEVICE_TABLE* in a very simple way — it informs the kernel that this driver is to be used if it finds any PCI device with the mentioned vendor and device IDs. When the kernel finds the device, it calls the *probe callback* function; and if it sees that the device is removed, then it will call the *remove callback* function. A detailed discussion on PCI drivers is outside the scope of this article.

```
static int ourfb_pci_init(struct pci_dev *dev, const struct
pci_device_id *ent)
{
    struct fb_info *info;
    struct ourfb_par *par;
    struct device *device = &dev->dev;
    int cmap_len, retval;
    info = framebuffer_alloc(sizeof(struct ourfb_par), device);
    if (!info) {
         return -ENOMEM;
    }
    par = info->par;
    info->screen_base = framebuffer_virtual_memory;
    info->fbops = &ourfb_ops;
    info->fix = ourfb_fix;
    info->pseudo_palette = pseudo_palette;
    info->flags = FBINFO_DEFAULT;
    if (fb_alloc_cmap(&info->cmap, cmap_len, 0))
     return -ENOMEM;
    if (register_framebuffer(info) < 0) {
     fb_dealloc_cmap(&info->cmap);
     return -EINVAL;
    }
    pci_set_drvdata(dev, info);
    return 0;
}
static void ourfb_pci_remove(struct pci_dev *dp)
{
     struct fb_info *p = pci_get_drvdata(dp);
     unregister_framebuffer(p);
     fb_dealloc_cmap(&p->cmap);
```